

PandOSsh — Fase 2

Il Nucleus: inizializzazione, scheduler, eccezioni, interrupt
Guida dettagliata riga per riga, funzione per funzione

Materiale di studio per l'interrogazione

A.A. 2025/2026

Indice

1	Introduzione generale alla Fase 2	1
2	Vocabolario hardware necessario	1
2.1	Le primitive della libreria <code>uriscv/liburiscv.h</code>	2
3	<code>phase2/headers/globals.h</code> — le variabili globali del Nucleus	3
4	<code>phase2/debug.h</code> — macro di debug condizionale	4
5	<code>phase2/initial.c</code> — inizializzazione del sistema	5
5.1	Variabili globali definite qui	5
5.2	<code>main</code> — la funzione di bootstrap	5
6	<code>phase2/scheduler.c</code> — lo scheduler	7
6.1	Passo 0 — gestione dello <code>YIELD</code>	8
6.2	Passo 1 — dispatch di un processo pronto	8
6.3	Passi 2–4 — il sistema è “fermo”	8
7	<code>phase2/exceptions.c</code> — eccezioni e system call	9
7.1	Funzioni di utilità (<code>static</code>)	9
7.2	<code>terminateProcess</code> — terminazione ricorsiva	10
7.3	<code>uTLB_RefillHandler</code> — TLB-Refill event	12
7.4	<code>tlbExceptionHandler</code> e <code>programTrapHandler</code>	12
7.5	<code>exceptionHandler</code> — il dispatcher principale	13
7.6	<code>syscallHandler</code> — le 10 syscall del Nucleus	14
7.6.1	Protezione dei privilegi	14
7.6.2	<code>SYS1</code> — <code>CREATEPROCESS</code>	14
7.6.3	<code>SYS2</code> — <code>TERMPROCESS</code>	15
7.6.4	<code>SYS3</code> — <code>PASSEREN (P)</code>	16
7.6.5	<code>SYS4</code> — <code>VERHOGEN (V)</code>	16
7.6.6	<code>SYS5</code> — <code>DOIO</code>	16
7.6.7	<code>SYS6</code> — <code>GETTIME</code>	17
7.6.8	<code>SYS7</code> — <code>CLOCKWAIT</code>	17
7.6.9	<code>SYS8</code> — <code>GETSUPPORTPTR</code>	18
7.6.10	<code>SYS9</code> — <code>GETPROCESSID</code>	18
7.6.11	<code>SYS10</code> — <code>YIELD</code>	18
7.7	<code>passUpOrDie</code> — il meccanismo di pass-up-or-die	18

8	phase2/interrupts.c — gestione degli interrupt	19
8.1	Costanti e utility	19
8.2	interruptHandler — struttura generale	19
8.3	PLT — fine time slice (preemption)	20
8.4	Interval Timer — tick dello pseudo-clock	20
8.5	Interrupt di device (linee 3–7, <code>excCode</code> 17–21)	21
8.5.1	Il caso speciale del terminale (linea 7)	22
9	Il ciclo completo di una operazione di I/O — riepilogo narrativo	23
10	Il test driver ufficiale: phase2/p2test.c	23

1 Introduzione generale alla Fase 2

La **Phase 2** costruisce, sopra le strutture dati della Fase 1, il **Nucleus**: il kernel vero e proprio di PandOSsh. È il primo livello del sistema che gira in modo autonomo, gestendo il ciclo di vita completo dei processi, la loro schedulazione, le eccezioni (incluse le *system call*) e gli *interrupt*. Il Nucleus gira sempre in kernel-mode, con gli interrupt disabilitati nei tratti critici, e — come tutto il progetto — senza mai allocare memoria dinamicamente.

Il codice è distribuito su quattro moduli, ciascuno con una responsabilità precisa:

File	Ruolo
<code>initial.c</code>	Bootstrap: popola il Pass Up Vector, inizializza la Fase 1 e le strutture globali, crea il primo pr
<code>scheduler.c</code>	Politica di scheduling: round-robin con priorità, rilevamento del “sistema fermo” (<code>halt/wait/dea</code>
<code>exceptions.c</code>	Punto di ingresso di <i>tutte</i> le eccezioni sincrone: le 10 system call del Nucleus, i page fault, i pro
<code>interrupts.c</code>	Gestione degli interrupt asincroni: fine time-slice (PLT), tick dello pseudo-clock (Interval Timer

Idea chiave. Il Nucleus non sa *nulla* della memoria virtuale o delle U-proc della Fase 3: quando incontra un’eccezione che non sa gestire (page fault, program trap generico), la “passa su” al livello superiore tramite il meccanismo `passUpOrDie`, oppure termina il processo se questo non ha un livello superiore a cui rivolgersi. Questo disaccoppiamento è ciò che rende la Fase 3 costruibile *sopra* la Fase 2 *senza modificarla*.

2 Vocabolario hardware necessario

Kernel vs user mode

Il campo MPP nel registro `status` distingue kernel-mode (M, privilegiato: 0x1800) da user-mode (U: 0). Il Nucleus gira sempre in kernel-mode.

Eccezione vs interrupt

Entrambe deviano il flusso al gestore installato nel *Pass Up Vector*. Un’**eccezione** è sincrona (causata dall’istruzione corrente: `ecall`, accesso a memoria non valido). Un **interrupt** è asincrono (un device o un timer che segnala un evento). Si distinguono guardando il **bit 31** del registro `cause` salvato: se acceso, è un interrupt; altrimenti il campo basso (`ExcCode`) indica il tipo preciso di eccezione.

BIOSDATAPAGE

Indirizzo fisso (0x0FFFF000) dove il firmware salva lo stato completo del processore (`state_t`) nel momento in cui avviene un’eccezione/interrupt. Il Nucleus lo interpreta come `savedState`.

Pass Up Vector

Tabella fissa (0x0FFFF900) che dice al firmware *dove saltare* in caso di eccezione: contiene due coppie (indirizzo dell’handler, stack pointer da usare), una per il *TLB-Refill event* e una per tutte le altre eccezioni.

TOD / PLT / Interval Timer

TOD (Time Of Day) è l’orologio assoluto, letto con `STCK`, usato per misurare il tempo CPU. Il **PLT** (Processor Local Timer) è il timer che genera l’interrupt di fine time-slice (preemption), impostato con `setTIMER`. L’**Interval Timer** è il timer di sistema che batte lo *pseudo-clock* ogni PSECOND (100 ms), riarmato con `LDIT`.

2.1 Le primitive della libreria `uriscv/liburiscv.h`

Tutte le funzioni in MAIUSCOLO (`STCK`, `LDST`, ...) che compaiono nel codice dei tre documenti **non sono funzioni del progetto**: sono primitive fornite dalla SDK dell’emulatore (`uriscv/liburiscv.h`), implementate in assembly, che eseguono un’istruzione macchina privilegiata o accedono direttamente ai registri del processore. Vale la pena avere un riferimento unico di cosa fa ciascuna, dato che vengono usate in continuazione senza ulteriori spiegazioni nei listati:

Primitiva	Categoria	Cosa fa
<code>SYSCALL(n, a1, a2, a3)</code>	<code>syscall</code>	Esegue l’istruzione <code>ecall</code> con <code>n</code> in <code>a0</code> e gli argomenti in <code>a1-a3</code> : genera l’eccezione che porta a <code>syscallHandler</code> (Nucleus) o <code>supSyscallHandler</code> (Support Level). Ritorna il valore lasciato in <code>a0</code> dal gestore. È l’ <i>unico</i> modo con cui un processo comunica con un livello sottostante.
<code>STCK(v)</code>	<code>tempo</code>	Legge il TOD (Time-Of-Day) corrente nella variabile <code>v</code> : la base di ogni misura di tempo (contabilità CPU, <code>GETTIME</code> , <code>CLOCKWAIT</code>).
<code>STST(statep)</code>	<code>stato</code>	<i>Store State</i> : salva lo stato <i>corrente</i> del processore in <code>*statep</code> . Usata solo dal test driver <code>p2test.c</code> per catturare un template di stato da cui partire nel costruire gli stati iniziali dei processi di test.
<code>LDST(statep)</code>	<code>stato</code>	<i>Load State</i> : carica <code>*statep</code> nel processore e riprende l’esecuzione da lì — non ritorna mai al chiamante. È l’istruzione con cui lo scheduler avvia/riprende un processo, e con cui ogni gestore di eccezione “chiude” la gestione restituendo il controllo.
<code>LDCXT(sp, status, pc)</code>	<code>pass-up</code>	<i>Load Context</i> : come <code>LDST</code> , ma invece di un intero <code>state_t</code> prende separatamente i tre valori di un <code>context_t</code> (si veda il documento di Fase 1) e salta lì — è il meccanismo con cui <code>passUpOrDie</code> trasferisce il controllo al Support Level.
<code>WAIT()</code>	<code>CPU</code>	Sospende la CPU a basso consumo finché non arriva un interrupt (usata dallo scheduler quando non c’è nulla di pronto ma qualcuno è soft-blocked).

PANIC()	arresto	Arresta la macchina virtuale segnalando un errore fatale/irrecuperabile (deadlock, invariante violato).
HALT()	arresto	Arresta la macchina virtuale in modo “pulito”, quando <code>processCount == 0</code> : è la conclusione normale del sistema.
setTIMER(v)	timer	Carica il valore <code>v</code> nel PLT (Processor Local Timer): allo scadere genera l'interrupt <code>excCode == 7</code> . Caricato con <code>NEVER</code> per disarmarlo.
LDIT(v)	timer	Carica il valore <code>v</code> nell'Interval Timer (riarmato a ogni tick con <code>PSECOND</code> , cioè ogni 100 ms).
getStatus() / setStatus(v)	registri	Leggono/scrivono il registro <code>status</code> <i>corrente</i> del processore (diverso da <code>state_t->status</code> , che è una <i>copia salvata</i>): usate per abilitare/disabilitare bit specifici a runtime (es. il PLT prima di un <code>WAIT</code> , o gli interrupt attorno a un aggiornamento di TLB in Fase 3).
setMIE(v)	registri	Scrive la maschera delle sorgenti di interrupt abilitate (es. <code>MIE_ALL & ~MIE_MIE_MASK</code> per abilitare tutto tranne il PLT).
TLBCLR()	TLB	Invalida (azzerà) tutte le entry del TLB.
TLBWR()	TLB	<i>TLB Write Random</i> : scrive una nuova traduzione (da <code>ENTRYHI/ENTRYLO</code>) in una entry del TLB scelta automaticamente dall'hardware. Usata sia dal TLB-Refill handler sia dal Pager (Fase 3) per installare una traduzione.
setENTRYHI(v) / setENTRYLO(v)	TLB	Impostano i registri di appoggio <code>EntryHI/EntryLO</code> <i>prima</i> di una <code>TLBWR</code> : rispettivamente <code>VPN+ASID</code> e <code>PFN+bit</code> di controllo (<code>V, D</code>) della traduzione da scrivere.

Nessuna di queste primitive è definita nel codice del progetto: sono dichiarate `extern` in `uriscv/liburiscv.h` e la loro implementazione reale (poche istruzioni assembly ciascuna) vive nell'installazione della SDK dell'emulatore, non nei sorgenti consegnati.

3 phase2/headers/globals.h — le variabili globali del Nucleus

```

1 extern int processCount;
2 extern int softBlockCount;
3 extern struct list_head readyQueue;
4 extern pcb_t *currentProcess;
5 extern pcb_t *yieldedProcess;
6 extern pcb_t *activeProcs[MAXPROC];
7
8 #define DEV_SEM_BASE(line, dev) (((line) - 3) * 8 + (dev))
9 #define TERM_TX_SEM(dev) (32 + (dev))
10 #define TERM_RX_SEM(dev) (40 + (dev))
11 #define PSEUDOCLOCK_SEM 48
12 #define TOT_SEMS 49
13
14 extern int devSems[TOT_SEMS];
15 extern cpu_t startTOD;
```

Tutte queste variabili sono *definite* (con lo storage reale) in `initial.c` e dichiarate `extern` qui per essere condivise dagli altri tre moduli.

- **processCount**: numero di processi **vivi** (creati e non ancora terminati), in qualunque stato (pronto, in esecuzione, bloccato). Quando arriva a 0 significa che tutto il lavoro del sistema è concluso.
- **softBlockCount**: numero di processi bloccati in attesa di un evento *esterno asincrono* (un device o lo pseudo-clock). Serve allo scheduler per distinguere “sto aspettando un interrupt” da “sono in deadlock”.
- **readyQueue**: la coda dei processi pronti, ordinata per priorità (vedi `insertProcQ` in Fase 1).
- **currentProcess**: puntatore al PCB del processo in esecuzione in questo momento, oppure NULL se nessun processo sta girando (il Nucleus stesso è “tra” due processi).
- **yieldedProcess**: puntatore temporaneo usato per implementare la syscall `YIELD` (`SYS10`): tiene il processo che ha appena ceduto volontariamente la CPU, in modo che lo scheduler possa decidere se farlo ripartire subito o dargli priorità inferiore rispetto a un altro processo pronto di pari o maggiore priorità.
- **activeProcs[MAXPROC]**: array di puntatori a *tutti* i processi vivi (non solo quelli pronti). È una struttura **aggiunta rispetto alla specifica minima** di Pandos, introdotta per due motivi precisi: (1) la syscall `TERMPROCESS` deve poter trovare un processo *per PID* anche se questo è bloccato (quindi non nella `readyQueue`); (2) lo scheduler deve poter distinguere in modo affidabile la condizione di deadlock. Il costo è $O(\text{MAXPROC})$ per ricerca/inserimento/rimozione, accettabile dato il limite fisso di 20 processi.
- **devSems[49]**: array dei semafori dei device. I primi 48 elementi coprono le 5 linee di interrupt esterne (disk, flash, ethernet, printer, terminal) $\times$ 8 device ciascuna secondo la formula `DEV_SEM_BASE(line, dev) = (line-3)*8 + dev`; per i terminali, che hanno due sotto-canali indipendenti (trasmissione e ricezione), si usano indici dedicati `TERM_TX_SEM/TERM_RX_SEM` (che di fatto sovrascrivono/estendono lo spazio della linea terminale, indici 32–47). L’ultimo elemento, indice 48 (`PSEUDOCLK_SEM`), è il semaforo dello pseudo-clock.
- **startTOD**: valore del TOD registrato all’ultimo punto in cui si è iniziato a “contare” il tempo CPU del processo corrente (dispatch, syscall, interrupt): confrontato con il TOD attuale permette di calcolare quanti cicli attribuire a `p_time`.

4 phase2/debug.h — macro di debug condizionale

Prima di entrare nei quattro moduli veri e propri, vale la pena conoscere `debug.h`: è incluso da *tutti* (`initial.c`, `scheduler.c`, `exceptions.c`, `interrupts.c`) e definisce il meccanismo di stampa usato per tracciare l’esecuzione del Nucleus a runtime.

```

1 #define DEBUG_INIT 0
2 #define DEBUG_SCHED 0
3 #define DEBUG_EXC 0
4 #define DEBUG_INT 0
5
6 #define DEBUG_ENABLED 1
7
8 #if DEBUG_ENABLED
9 #define DELAY() for (volatile int i = 0; i < 60; i++)
10 static inline void debug_print(const char *msg) {
11     unsigned int *command = (unsigned int *) (0x10000254 + 3*4);
12     while (*msg != '\0') {
13         *command = 2 | (((unsigned int)*msg) << 8);
14         DELAY();
15         msg++;
16     }
17 }
18 static inline void debug_hex(const char *label, unsigned int val) {
19     /* stampa label seguita dal valore in esadecimale a 8 cifre */
20 }
21 #else
22 #define debug_print(msg) ((void)0)

```

```

23 #define debug_hex(label, val) ((void)0)
24 #endif
25 #endif

```

- **Due livelli di controllo indipendenti:** un interruttore *globale* (DEBUG_ENABLED) e quattro interruttori *per modulo* (DEBUG_INIT, DEBUG_SCHED, DEBUG_EXC, DEBUG_INT). Ogni modulo, in testa al proprio file, ridefinisce le macro IDBG/SDBG/EDBG (con nome diverso solo per evitare collisioni tra moduli) in base al proprio flag: se attivo, espandono a `debug_print/debug_hex`; se disattivo, espandono a `((void)0)` (un'istruzione nulla, eliminata dal compilatore, così da non lasciare *alcun* costo a runtime quando il debug è spento). Nella versione consegnata tutti e quattro i flag di modulo sono a 0: il debug è disattivabile modulo per modulo senza ricompilare tutto il progetto, ma di default è silenzioso.
- `debug_print(msg)`: scrive una stringa **direttamente sui registri hardware del terminale 0** (indirizzo `0x10000254 + 3*4`, cioè il registro di comando della trasmissione), un carattere alla volta, con un comando PRINTCHR (2) composto a mano (carattere nel byte alto, opcode nel byte basso) — **non** passa mai da DOI0: è pensato per stampare diagnostica *anche prima* che il Nucleus sia in grado di gestire syscall e interrupt correttamente (utile per debuggare proprio `initial.c` durante il boot), o comunque senza dipendere dal meccanismo che si starebbe cercando di diagnosticare.
- `DELAY()`: un ciclo di attesa attiva (*busy-wait*) da 60 iterazioni tra un carattere e l'altro, necessario perché la scrittura sul registro di comando non è istantanea: senza una pausa minima, caratteri scritti troppo rapidamente in successione andrebbero persi (nessun meccanismo di sincronizzazione reale è coinvolto, è un semplice ritardo empirico).
- `debug_hex(label, val)`: come `debug_print`, ma stampa anche un valore a 32 bit convertito manualmente in 8 cifre esadecimali (utile per stampare indirizzi di semafori, PID, valori di registri senza dover comporre a mano la stringa a ogni chiamata).

È lo strumento di debug *effettivamente usato* durante lo sviluppo del Nucleus (a differenza di `klog.c`, discusso nel documento di Fase 1, che è compilato ma mai invocato): le chiamate a `IDBG_HEX`, `SDBG`, `EDBG_HEX`, ecc. visibili nei listati dei moduli successivi sono per l'appunto invocazioni di queste macro.

5 phase2/initial.c — inizializzazione del sistema

5.1 Variabili globali definite qui

```

1 int          processCount;
2 int          softBlockCount;
3 struct list_head readyQueue;
4 pcb_t        *currentProcess;
5 int          devSems[TOT_SEMS];
6 cpu_t        startTOD;
7 pcb_t        *activeProcs[MAXPROC];
8 pcb_t        *yieldedProcess = NULL;

```

Questo è l'*unico* punto del progetto in cui queste variabili vengono effettivamente allocate (storage reale); ovunque altrove sono `extern`.

5.2 main — la funzione di bootstrap

```

1 int main(void) {
2     passupvector_t *passUpVec = (passupvector_t *) PASSUPVECTOR;
3     memaddr ramtop;
4     RAMTOP(ramtop);
5
6     passUpVec->tlb_refill_handler = (memaddr) uTLB_RefillHandler;
7     passUpVec->tlb_refill_stackPtr = ramtop;
8     passUpVec->exception_handler = (memaddr) exceptionHandler;
9     passUpVec->exception_stackPtr = ramtop - PAGE_SIZE;

```

```

10
11     initPcb();
12     initASL();
13
14     processCount    = 0;
15     softBlockCount = 0;
16     currentProcess = NULL;
17     mkEmptyProcQ(&readyQueue);
18
19     for (int i = 0; i < TOT_SEMS; i++) devSems[i] = 0;
20
21     STCK(startTOD);
22
23     for (int i = 0; i < MAXPROC; i++)
24         activeProcs[i] = NULL;
25
26     LDIT(PSECOND);
27
28     pcb_t *testPcb = allocPcb();
29     if (testPcb == NULL) PANIC();
30
31     testPcb->p_s.status      = MSTATUS_MIE_MASK | MSTATUS_MPIE_MASK | MSTATUS_MPP_M;
32     testPcb->p_s.mie        = MIE_ALL;
33     testPcb->p_s.reg_sp      = ramtop - (2 * PAGESIZE);
34     testPcb->p_s.pc_epc      = (memaddr) test;
35     testPcb->p_s.prio        = PROCESS_PRIO_LOW;
36
37     activeProcs[0] = testPcb;
38     insertProcQ(&readyQueue, testPcb);
39     processCount = 1;
40
41     scheduler();
42     return 0;
43 }

```

Passo per passo:

1. **Calcolo di ramtop.** La macro `RAMTOP(ramtop)` legge dai registri di configurazione della macchina (`RMBASEADDR`, `RMBASESIZE`) l'indirizzo del *primo byte oltre la fine della RAM* disponibile.
2. **Popolamento del Pass Up Vector.** Il Nucleus registra se stesso come gestore *unico* di tutte le eccezioni hardware:
 - `exception_handler = exceptionHandler`: qualunque eccezione “normale” (syscall, program trap, page fault non-refill) salta a `exceptionHandler` (in `exceptions.c`);
 - `tlb_refill_handler = uTLB_RefillHandler`: il caso speciale di TLB-Refill (percorso veloce, vedi Fase 3) salta a un handler dedicato;
 - **scelta implementativa importante**: gli stack di questi due gestori sono impostati sui *due frame più alti della RAM* (`ramtop` per il TLB-Refill, `ramtop - PAGESIZE` per le eccezioni), *non* su un unico indirizzo fisso condiviso (`KERNELSTACK`). Motivo: se si usasse un solo stack, un'eccezione che avviene *durante* la gestione di un'altra (eccezioni annidate, possibili perché nulla vieta a un TLB-Refill di avvenire mentre gira l'exception handler) sovrascriverebbe lo stack dell'altro handler, corrompendone lo stato. Isolarli su due frame fisici separati elimina questo rischio.
3. **Inizializzazione della Fase 1.** `initPcb()` e `initASL()` preparano le free list dei PCB e dei descrittori di semaforo (vedi documento di Fase 1).
4. **Azzeramento delle strutture globali del Nucleus:** `processCount`, `softBlockCount`, `currentProcess = NULL`, `readyQueue` vuota, tutti i 49 `devSems` a 0 (semantica a contatore: valore iniziale 0, non negativo, perché nessun processo è ancora bloccato su di essi).
5. **Lettura del TOD reale** (`STCK(startTOD)`) invece di azzerare `startTOD`. Se si azzerasse, il primo processo creato “erediterebbe” come tempo CPU l'intero intervallo trascorso dall'avvio della macchina virtuale fino a questo punto del bootstrap (che può essere non trascurabile), falsando la contabilità di `GETTIME`.

6. `activeProcs[i] = NULL` per tutti gli slot: nessun processo vivo ancora.
7. **Avvio dell'Interval Timer:** `LDIT(PSECOND)` programma il primo tick dello pseudo-clock tra 100 000 microsecondi (100 ms, `PSECOND` in `const.h`).
8. **Creazione del processo iniziale** ('test', il punto di ingresso della Fase 2/3 — in Fase 2 puro esegue i test P2, in Fase 3 è l'`InstantiatorProcess`): si alloca un PCB con `allocPcb()` (fallimento → `PANIC()`), situazione impossibile dato che la free list è appena stata popolata con 20 slot), e si configura manualmente lo stato iniziale del processore:
 - **status:** interrupt abilitati (`MIE`), interrupt “precedenti” abilitati (`MPIE`, ripristinati automaticamente al primo `LDST`), `MPP = kernel-mode` (`MPP_M`) — il processo test gira in kernel-mode, non è una U-proc;
 - **mie = `MIE_ALL`:** tutte le sorgenti di interrupt abilitate a livello di maschera;
 - **reg_sp = `ramtop - 2*PAGESIZE`:** lo stack del processo test è posizionato *sotto* i due frame riservati agli handler del kernel (che occupano `ramtop` e `ramtop-PAGESIZE`), evitando ogni sovrapposizione;
 - **pc_epc = (`memaddr`) test:** il processo parte eseguendo la funzione `test` (definita in `p2test.c` per la Fase 2 pura, o in `phase3/initProc.c` per la Fase 3, a seconda del target di compilazione);
 - **p_prio = `PROCESS_PRIO_LOW`:** priorità bassa (0).

Il PCB viene poi registrato in `activeProcs[0]`, inserito nella `readyQueue`, e `processCount` viene impostato a 1.

9. **Avvio dello scheduler:** `scheduler()` non ritorna mai (in condizioni normali termina l'esecuzione con `HALT()`); la riga `return 0`; successiva è dead code raggiunto solo in caso di bug grave.

Da notare la scelta di *non* duplicare in `main` la logica di selezione “chi eseguire per primo”: si crea un solo processo pronto e si lascia allo `scheduler()` il compito di estrarlo e avviarlo, mantenendo tutta la logica di dispatch in un unico posto.

6 phase2/scheduler.c — lo scheduler

```

1 void scheduler(void) {
2
3     if (yieldedProcess != NULL) {
4         pcb_t *y = yieldedProcess;
5         yieldedProcess = NULL;
6         if (!emptyProcQ(&readyQueue) &&
7             headProcQ(&readyQueue)->p_prio >= y->p_prio) {
8             insertProcQ(&readyQueue, y);
9         } else {
10            currentProcess = y;
11            STCK(startTOD);
12            setTIMER(TIMESLICE * (((cpu_t *) TIMESCALEADDR)));
13            LDST(&currentProcess->p_s);
14        }
15    }
16
17    if (!emptyProcQ(&readyQueue)) {
18        currentProcess = removeProcQ(&readyQueue);
19        STCK(startTOD);
20        setTIMER(TIMESLICE * (((cpu_t *) TIMESCALEADDR)));
21        LDST(&currentProcess->p_s);
22    }
23
24    currentProcess = NULL;
25
26    if (processCount == 0) {
27        HALT();
28    }
29

```

```

30  if (softBlockCount > 0) {
31      setMIE(MIE_ALL & ~MIE_MTIE_MASK);
32      unsigned int status = getSTATUS();
33      status |= MSTATUS_MIE_MASK;
34      setSTATUS(status);
35      WAIT();
36  }
37
38  PANIC();
39 }

```

La politica implementata è un **round-robin con priorità**: tra i processi pronti si sceglie sempre il più prioritario (grazie all’ordinamento già garantito da `insertProcQ` in Fase 1); a parità di priorità, l’ordine di arrivo (FIFO) decide chi va per primo, e ciascuno gira per al massimo una *time slice* prima di essere rimesso in coda (dal gestore dell’interrupt PLT).

6.1 Passo 0 — gestione dello YIELD

Se `yieldedProcess` non è NULL (un processo ha appena eseguito la syscall `YIELD`, `SYS10`), lo scheduler decide se rimetterlo *immediatamente* in coda oppure farlo ripartire subito:

- se la ready queue *non* è vuota e il processo in testa ha priorità $\geq$ a quella dello yielder $\rightarrow$ lo yielder *cede davvero*: viene reinserito in coda con `insertProcQ` (che lo posizionerà secondo la sua priorità), e si prosegue al passo successivo (che estrarrà dalla coda il processo più prioritario, possibilmente lo stesso yielder se nel frattempo nessuno lo ha superato — ma tipicamente un altro);
- altrimenti (coda vuota, oppure lo yielder è comunque il più prioritario disponibile) `currentProcess` diventa di nuovo lo yielder: si registra il TOD, si riarma il PLT con una nuova time slice completa, e si riparte con `LDST`. *Senza* questo ramo, uno `YIELD` in un sistema con un solo processo pronto risulterebbe in un `removeProcQ` seguito da un nuovo giro immediato comunque, ma la gestione esplicita rende il comportamento uniforme ed evita una doppia ri-scansione della coda.

6.2 Passo 1 — dispatch di un processo pronto

Se la ready queue non è vuota, se ne estrae la testa (`removeProcQ`, $O(1)$, rispetta la priorità), si registra `startTOD` (per la contabilità del tempo CPU che seguirà), si carica nel PLT una nuova **time slice** pari a `TIMESLICE * TIMESCALEADDR` (`TIMESLICE = 5 000 000`, moltiplicato per il fattore di scala del TOD letto dal registro hardware `TIMESCALEADDR`, per ottenere il valore corretto nell’unità di tempo della macchina), e si riprende l’esecuzione del processo con `LDST` (Load State: ripristina tutti i registri e il PC dal `p_s` salvato, e non ritorna mai al chiamante — è un vero e proprio salto).

6.3 Passi 2–4 — il sistema è “fermo”

Se si arriva qui, non c’era nessun processo pronto da eseguire (`currentProcess` viene esplicitamente riportato a NULL): lo scheduler distingue tre possibili situazioni, in ordine:

1. `processCount == 0`: non esiste più nessun processo vivo, tutto il lavoro del sistema è concluso $\rightarrow$ `HALT()` ferma la macchina virtuale.
2. `softBlockCount > 0`: esistono processi vivi ma *tutti* sono in attesa di un evento esterno (I/O o pseudo-clock). Il Nucleus non ha nulla da eseguire ma *non* deve fermarsi: entra in `WAIT()` (istruzione che sospende la CPU a basso consumo finché non arriva un interrupt). Prima di farlo, abilita tutti gli interrupt **tranne** il PLT (`MIE_ALL & ~MIE_MTIE_MASK`): non ha senso ricevere un interrupt di fine time-slice quando nessun processo sta effettivamente “consumando” uno slice, e si evita così un interrupt PLT spurio (che complicherebbe inutilmente la gestione, dato che `currentProcess` sarebbe NULL).

3. **Altrimenti** (processi vivi, nessuno pronto, nessuno soft-blocked): è un vero **deadlock** — non esiste alcun evento futuro (né un dispatch, né un interrupt atteso) che possa far progredire il sistema. Si chiama `PANIC()`, che arresta la macchina segnalando l'errore.

7 phase2/exceptions.c — eccezioni e system call

Questo è il modulo più corposo del Nucleus. Gestisce: il TLB-Refill event (percorso condizionale per la Fase 3), il dispatch di tutte le eccezioni sincrone, l'implementazione delle 10 syscall del Nucleus, la terminazione ricorsiva dei processi, e il meccanismo di pass-up-or-die.

7.1 Funzioni di utilità (static)

```
1 static void copyState(state_t *dst, state_t *src) {
2     unsigned int *d = (unsigned int *) dst;
3     unsigned int *s = (unsigned int *) src;
4     for (int i = 0; i < STATE_T_SIZE_IN_BYTES / WORDLEN; i++)
5         d[i] = s[i];
6 }
```

`copyState`: copia byte-a-byte (in realtà word-a-word, 4 byte alla volta) l'intero contenuto di uno `state_t` da `src` a `dst`, trattando le due strutture come array di interi di dimensione `STATE_T_SIZE_IN_BYTES / WORDLEN` ($148/4 = 37$ word). È usata al posto di un'assegnazione diretta `*dst = *src` probabilmente per compatibilità/chiarzza esplicita a livello di word; il risultato è equivalente. Serve ogni volta che occorre salvare lo stato del processore letto da `BIOSDATAPAGE` dentro il PCB del processo corrente (`currentProcess->p_s`), tipicamente prima di bloccare o rischedulare il processo.

```
1 static void updateCPUTime(void) {
2     if (currentProcess) {
3         cpu_t now;
4         STCK(now);
5         currentProcess->p_time += now - startTOD;
6         startTOD = now;
7     }
8 }
```

`updateCPUTime`: implementa la **contabilità del tempo CPU**. A ogni “punto critico” (ingresso in una syscall, in un interrupt, in un blocco) si legge il TOD corrente (`STCK`), si calcola il delta rispetto all'ultimo `startTOD` registrato, lo si somma a `p_time` del processo corrente, e si aggiorna `startTOD` al nuovo valore. Questo schema, applicato sistematicamente in *ogni* punto in cui il controllo lascia il processo corrente (anche solo temporaneamente, come durante la gestione di una syscall), garantisce che nessun ciclo di CPU vada perso o venga contato due volte, anche in presenza di eccezioni annidate o di più syscall consecutive nello stesso quanto di tempo.

```
1 static int isDeviceSemaphore(int *semAdd) {
2     int *base = &devSems[0];
3     int *top = &devSems[TOT_SEMS];
4     if (semAdd < base || semAdd >= top) return 0;
5     int idx = (int)(semAdd - base);
6     if (idx == PSEUDOCLOCK_SEM) return 0;
7     return 1;
8 }
```

`isDeviceSemaphore`: distingue, dato l'indirizzo di un semaforo, se è un semaforo di *device* (vero) oppure un semaforo “normale” (di sincronizzazione tra processi, oppure lo pseudo-clock: falso). Verifica prima che l'indirizzo cada dentro l'array `devSems` (aritmetica dei puntatori: confronto tra indirizzi), poi esclude esplicitamente l'ultimo indice (`PSEUDOCLOCK_SEM`), che pur essendo fisicamente dentro `devSems` rappresenta lo pseudo-clock, contato separatamente. Questa distinzione è cruciale per `blockCurrentProcess`: il contatore `softBlockCount` deve incrementare per i device *ma anche* per lo pseudo-clock (che infatti viene gestito a parte, incrementando esplicitamente in `CLOCKWAIT`) — la funzione qui serve invece a *escludere* lo pseudo-clock dal doppio conteggio quando ci si blocca tramite il percorso generico `blockCurrentProcess`.

```

1 static void blockCurrentProcess(int *sem) {
2     if (!currentProcess) PANIC();
3     cpu_t now;
4     STCK(now);
5     currentProcess->p_time += now - startTOD;
6     startTOD = now;
7     currentProcess->p_semAdd = sem;
8     if (isDeviceSemaphore(sem)) {
9         softBlockCount++;
10    }
11    insertBlocked(sem, currentProcess);
12    currentProcess = NULL;
13    scheduler();
14 }

```

blockCurrentProcess: centralizza la logica di blocco di **currentProcess** su un semaforo qualsiasi. Aggiorna il tempo CPU (stessa logica di **updateCPUtime**, duplicata inline), imposta **p_semAdd**, incrementa **softBlockCount** *solo* se il semaforo è un semaforo di device reale (esclude quindi sia i semafori “normali” di sincronizzazione sia — tramite **isDeviceSemaphore** — lo pseudo-clock, il cui incremento è gestito esplicitamente dal chiamante quando serve, es. in **CLOCKWAIT**), inserisce il processo nella ASL con **insertBlocked**, azzerava **currentProcess** e richiama lo scheduler. È la funzione invocata da **PASSEREN** e da **DOIO** quando il processo deve effettivamente bloccarsi.

```

1 static void activeProcs_add(pcb_t *p) {
2     for (int i = 0; i < MAXPROC; i++) {
3         if (activeProcs[i] == NULL) { activeProcs[i] = p; return; }
4     }
5     PANIC();
6 }
7
8 static void activeProcs_remove(pcb_t *p) {
9     for (int i = 0; i < MAXPROC; i++) {
10        if (activeProcs[i] == p) { activeProcs[i] = NULL; return; }
11    }
12 }
13
14 static pcb_t *findProcessByPid(int target) {
15     for (int i = 0; i < MAXPROC; i++) {
16         if (activeProcs[i] != NULL && activeProcs[i]->p_pid == target)
17             return activeProcs[i];
18     }
19     return NULL;
20 }

```

Tre funzioni di gestione dell’array **activeProcs**, tutte $O(\text{MAXPROC})$ per scansione lineare:

- **activeProcs_add(p)**: trova il primo slot libero (NULL) e vi inserisce p. Se non trova spazio (impossibile in condizioni normali, dato che **allocPcb** fallirebbe prima se i 20 PCB sono esauriti) chiama **PANIC()** per difesa.
- **activeProcs_remove(p)**: trova lo slot che contiene esattamente p e lo azzerava.
- **findProcessByPid(target)**: scansione lineare per trovare il processo (vivo, non necessariamente pronto) con un dato PID; ritorna NULL se non esiste (PID già terminato o mai esistito). Usata da **TERMPROCESS** quando il target non è il processo corrente.

7.2 terminateProcess — terminazione ricorsiva

```

1 static void terminateProcess(pcb_t *p) {
2     if (p == NULL) return;
3     if (p->p_pid == -1) return;
4
5     p->p_pid = -1;
6
7     pcb_t *child;
8     while ((child = removeChild(p)) != NULL) {
9         terminateProcess(child);
10    }

```

```

11 outChild(p);
12 activeProcs_remove(p);
13
14
15 if (p->p_semAdd != NULL) {
16     int *sem = p->p_semAdd;
17     pcb_t *removed = outBlocked(p);
18     p->p_semAdd = NULL;
19     if (removed != NULL) {
20         if (isDeviceSemaphore(sem) || sem == &devSems[PSEUDOCLOCK_SEM]) {
21             softBlockCount--;
22         } else {
23             (*sem)++;
24         }
25     }
26 }
27
28 outProcQ(&readyQueue, p);
29
30 if (p == currentProcess) {
31     currentProcess = NULL;
32 }
33
34 freePcb(p);
35 processCount--;
36 }

```

Implementa la semantica di **TERMPROCESS** (SYS2): terminare un processo significa terminare *ricorsivamente tutto il sotto-albero* radicato in esso (i figli, i figli dei figli, ecc.), come richiesto dalla specifica di Pandos. Analisi:

1. **Guardia contro doppie terminazioni:** se p è NULL o ha già $p_pid == -1$ (segnale interno “sto già terminando/terminato”), ritorna subito. Questo flag protegge da eventuali riferimenti circolari o rientri accidentali durante la ricorsione.
2. $p \rightarrow p_pid = -1$: marca p come “in fase di terminazione” *prima* di ricorrere sui figli, per evitare che una struttura anomala porti a rivisitarlo.
3. **Ricorsione sui figli:** il ciclo `while ((child=removeChild(p)) != NULL)` stacca ripetutamente il *primo* figlio di p e lo termina ricorsivamente; poiché `removeChild` accorcia la lista dei figli a ogni chiamata, il ciclo termina quando p non ha più figli. Questa è una visita in stile *depth-first*: prima si smaltisce interamente un ramo (e i suoi discendenti), poi si passa al fratello successivo.
4. `outChild(p)`: stacca p stesso dalla lista dei figli del *proprio* padre. Necessario perché, come spiegato in Fase 1, senza questo passo un processo che termina se stesso (o che viene terminato senza che il padre stia già iterando su di lui) resterebbe agganciato come “figlio fantasma”.
5. `activeProcs_remove(p)`: toglie p dall’array globale dei processi vivi.
6. **Se p era bloccato su un semaforo** ($p \rightarrow p_semAdd \neq \text{NULL}$): va tolto dalla coda di quel semaforo (`outBlocked`) e — punto delicato — bisogna decidere *cosa fare del valore del semaforo*:
 - se è un semaforo di device *oppure* lo pseudo-clock: il processo era “soft-blocked”, cioè aspettava un evento che *arriverà comunque* dall’hardware (l’interrupt del device o il prossimo tick). Il valore del semaforo **non** va toccato: sarà l’interrupt (quando arriverà) a fare la sua V regolare. Si aggiorna solo il contatore `softBlockCount-` (un processo soft-blocked in meno).
 - altrimenti (semaforo “normale”, di sincronizzazione tra processi): la P che il processo terminato aveva eseguito *va annullata* incrementando il semaforo $(*sem)++$, altrimenti il suo valore resterebbe sbilanciato verso il basso e un futuro processo che tenta la P si bloccherebbe per sempre aspettando un rilascio che non arriverà mai (il processo che avrebbe dovuto fare la V corrispondente non esiste più).
7. `outProcQ(&readyQueue, p)`: rimozione difensiva dalla ready queue (se p non vi si trova, la funzione non fa nulla e ritorna NULL senza errori: si veda Fase 1).

8. Se `p` era `currentProcess`, lo si azzerava esplicitamente: il chiamante (`TERMPROCESS` in `syscallHandler`) userà questo segnale per capire se deve richiamare direttamente lo scheduler.
9. `freePcb(p)`: il PCB torna alla free list della Fase 1.
10. `processCount--`: un processo vivo in meno.

7.3 uTLB_RefillHandler — TLB-Refill event

```

1 void uTLB_RefillHandler(void) {
2     state_t *savedState = (state_t *) BIOSDATAPAGE;
3
4     #ifdef SUPPORT_LEVEL
5         unsigned int vpn = (savedState->entry_hi >> VPNSHIFT) & 0xFFFFF;
6         int index;
7         if (vpn == 0xBFFFF) index = 31;
8         else index = (int)(vpn - 0x80000);
9
10        pteEntry_t *pte = &currentProcess->p_supportStruct->sup_privatePgTbl[index];
11        setENTRYHI(pte->pte_entryHI);
12        setENTRYLO(pte->pte_entryLO);
13        TLBWR();
14        LDST(savedState);
15    #else
16        setENTRYHI(0x80000000);
17        setENTRYLO(0x00000000);
18        TLBWR();
19        LDST(savedState);
20    #endif
21 }

```

Compilato *condizionalmente* in base alla macro `SUPPORT_LEVEL` (definita solo quando si compila il target `phase3`). Nella Fase 2 pura non esiste memoria virtuale a livello utente: il ramo `#else` è solo uno skeleton placeholder ereditato dal tester (scrive un'entry fissa per la pagina di test `p5`), mai realmente esercitato in modo significativo. Nella Fase 3 (`#ifdef SUPPORT_LEVEL`) è il **percorso veloce** dei page fault: quando la traduzione manca dal TLB *ma è già presente* nella Page Table privata della U-proc (`currentProcess->p_supportStruct->sup_privatePgTbl`), questo handler si limita a copiarla dentro il TLB (`setENTRYHI/ setENTRYLO/TLBWR`) e a riprendere l'esecuzione (`LDST`), senza mai coinvolgere il Pager né il disco. La discussione approfondita di questo meccanismo (e della distinzione rispetto al Pager) è nel documento di Fase 3.

7.4 tlbExceptionHandler e programTrapHandler

```

1 static void tlbExceptionHandler(void) {
2     passUpOrDie(PGFAULTEXCEPT);
3 }
4
5 static void programTrapHandler(void) {
6     passUpOrDie(GENERALEXCEPT);
7 }

```

Due semplici wrapper: qualunque eccezione di gestione memoria (page fault) viene delegata a `passUpOrDie` con tipo `PGFAULTEXCEPT` (0); qualunque altra eccezione non gestita direttamente dal Nucleus (istruzione illegale, misallineamento, ecc.) con tipo `GENERALEXCEPT` (1). Questi due indici corrispondono esattamente agli slot `[0]/[1]` degli array `sup_exceptState/sup_exceptContext` nella `support_t` (Fase 1/3): è così che il Nucleus, senza sapere nulla del Support Level, riesce comunque a smistare l'eccezione al gestore corretto di livello superiore.

7.5 exceptionHandler — il dispatcher principale

```

1 void exceptionHandler(void) {
2     state_t *savedState = (state_t *) BIOSDATAPAGE;
3     unsigned int cause = savedState->cause;
4 }

```

```

5  updateCPUTime();
6
7  if (cause & 0x80000000) {
8      interruptHandler();
9      return;
10 }
11
12 unsigned int excCode = cause & CAUSE_EXCCODE_MASK;
13
14 switch (excCode) {
15     case EXC_ECU:
16     case EXC_ECM:
17         savedState->pc_epc += WORDLEN;
18         syscallHandler(savedState);
19         break;
20
21     case EXC_IPF:
22     case EXC_LPF:
23     case EXC_SPF:
24     case EXC_MOD:
25     case EXC_TLBL:
26     case EXC_TLBS:
27     case EXC_UTLBL:
28     case EXC_UTLBS:
29         tlbExceptionHandler();
30         break;
31
32     default:
33         programTrapHandler();
34         break;
35 }
36 }

```

È l'unico punto di ingresso registrato nel Pass Up Vector per tutte le eccezioni non-refill. Sequenza:

1. Legge `savedState` da `BIOSDATAPAGE` e il registro `cause` salvato dal firmware.
2. `updateCPUTime()`: aggiorna *subito* la contabilità del tempo CPU del processo appena interrotto, prima di fare qualunque altra cosa — così anche il tempo trascorso mentre si decide cosa fare dell'eccezione viene attribuito correttamente.
3. **Bit 31 di cause acceso** → **interrupt**: delega immediatamente a `interruptHandler()` (Fase 2, modulo `interrupts.c`) e ritorna (l'interrupt handler non ritornerà mai qui: fa sempre LDST o chiama lo scheduler).
4. Altrimenti, legge il campo basso `excCode` e smista:
 - `EXC_ECU` (8, `ecall` da user-mode) o `EXC_ECM` (11, `ecall` da kernel-mode, usata dal Nucleus stesso e dai processi kernel come `test`): **avanza il PC di una word** (`WORDLEN=4`) *prima* di gestire la syscall. Questo è essenziale: senza avanzare il PC, quando il processo verrà ripreso rieseguirebbe la stessa istruzione `ecall` all'infinito. Poi chiama `syscallHandler(savedState)`.
 - un insieme di codici legati a page fault/TLB (`EXC_IPF`, `EXC_LPF`, `EXC_SPF` — i codici RISC-V standard per Instruction/Load/Store Page Fault — più i codici specifici dell'architettura μ RISCV `EXC_MOD`, `EXC_TLBL`, `EXC_TLBS`, `EXC_UTLBL`, `EXC_UTLBS`): tutti trattati uniformemente come “memory management trap” → `tlbExceptionHandler()`.
 - **default**: qualunque altra causa (istruzione illegale, accesso non allineato, breakpoint, ecc.) → `programTrapHandler()`.

7.6 syscallHandler — le 10 syscall del Nucleus

```

1  static void syscallHandler(state_t *savedState) {
2      int sysCode = (int) savedState->reg_a0;
3
4      if ((savedState->status & MSTATUS_MPP_MASK) == 0 && sysCode < 0) {
5          savedState->cause = PRIVINSTR;
6          programTrapHandler();

```

```

7      return;
8  }
9
10     if (sysCode >= 1) {
11         passUpOrDie(GENERALEXCEPT);
12         return;
13     }
14
15     switch (sysCode) {
16         case CREATEPROCESS: { ... }
17         case TERMPROCESS:   { ... }
18         case PASSEREN:      { ... }
19         case VERHOGEN:      { ... }
20         case DOIO:          { ... }
21         case GETTIME:       { ... }
22         case CLOCKWAIT:     { ... }
23         case GETSUPPORTPTR: { ... }
24         case GETPROCESSID:  { ... }
25         case YIELD:         { ... }
26         default:
27             passUpOrDie(GENERALEXCEPT);
28     }
29 }

```

7.6.1 Protezione dei privilegi

Il numero di syscall arriva in `a0` (convenzione RISC-V: `reg_a0`). Le syscall del Nucleus sono numerate con valori **negativi** da -1 (`CREATEPROCESS`) a -10 (`YIELD`); i valori **positivi** sono riservati alle syscall del Support Level (Fase 3). Prima regola:

```
if ((savedState->status & MSTATUS_MPP_MASK) == 0 && sysCode < 0)
```

Se il campo MPP dello `status` salvato è 0 (cioè il chiamante *era* in **user-mode**) e il codice richiesto è negativo (una syscall riservata al kernel), l'eccezione viene forzatamente riqualificata come `PRIVINSTR` (istruzione privilegiata, tentativo di uso non autorizzato) e delegata a `programTrapHandler()`: nessuna U-proc può invocare direttamente una syscall del Nucleus.

Seconda regola: se `sysCode >= 1` (syscall del Support Level, di competenza di un livello *superiore* al Nucleus, che non sa nulla di come gestirle), viene delegata con `passUpOrDie(GENERALEXCEPT)`: il Nucleus non implementa mai syscall positive, le passa sempre su.

7.6.2 SYS1 — CREATEPROCESS

```

1  case CREATEPROCESS: {
2      state_t *newState = (state_t *) savedState->reg_a1;
3      int prio = (int) savedState->reg_a2;
4      support_t *support = (support_t *) savedState->reg_a3;
5
6      pcb_t *child = allocPcb();
7      if (!child) {
8          savedState->reg_a0 = (unsigned int) -1;
9          copyState(&currentProcess->p_s, savedState);
10         LDST(&currentProcess->p_s);
11     }
12
13     copyState(&child->p_s, newState);
14     child->p_supportStruct = support;
15     child->p_prio = prio;
16
17     activeProcs_add(child);
18     insertChild(currentProcess, child);
19     processCount++;
20
21     savedState->reg_a0 = (unsigned int) child->p_pid;
22     copyState(&currentProcess->p_s, savedState);
23     insertProcQ(&readyQueue, currentProcess);
24     insertProcQ(&readyQueue, child);
25     currentProcess = NULL;

```

```

26     scheduler();
27     break;
28 }

```

Crea un nuovo processo figlio del chiamante. Argomenti (convenzione: **a1**, **a2**, **a3**): puntatore allo stato iniziale del processore che il nuovo processo dovrà avere, priorità, puntatore opzionale a una **support_t**. Passi:

1. **allocPcb()**: se la free list dei PCB è esaurita (limite di 20 processi vivi raggiunto), **a0** del chiamante viene impostato a -1 (fallimento) e si riprende immediatamente il chiamante con **LDST** — *niente* nuovo processo, nessuna modifica a **processCount**.
2. Altrimenti si copia lo stato iniziale fornito nel PCB del figlio (**copyState**), si registrano **support struct** e priorità.
3. Il figlio viene aggiunto ad **activeProcs** e inserito nell'albero come figlio di **currentProcess** (**insertChild**). **processCount++**.
4. **a0** del chiamante viene impostato al **PID del figlio** appena creato (valore di ritorno della syscall lato padre).
5. **Sia il padre sia il figlio** vengono inseriti nella ready queue (in ordine: prima il padre, poi il figlio, ma **insertProcQ** riordina comunque per priorità), e si richiama lo scheduler. Questo *abilita la preemption immediata*: se il figlio ha priorità superiore al padre, sarà lui il primo a essere estratto ed eseguito, anche prima che il padre riprenda dal punto della **ecall**.

7.6.3 SYS2 — TERMPROCESS

```

1  case TERMPROCESS: {
2      int targetPid = (int) savedState->reg_a1;
3      updateCPUTime();
4
5      pcb_t *target = (targetPid == 0) ? currentProcess : findProcessByPid(targetPid);
6      if (!target) {
7          LDST(savedState);
8      } else {
9          int terminatingSelf = (target == currentProcess);
10         terminateProcess(target);
11         if (terminatingSelf) currentProcess = NULL;
12     }
13
14     if (currentProcess == NULL) {
15         scheduler();
16     } else {
17         copyState(&currentProcess->p_s, savedState);
18         LDST(&currentProcess->p_s);
19     }
20     break;
21 }

```

Se **a1 == 0** il target è il processo chiamante stesso; altrimenti si cerca per PID con **findProcessByPid**. Se il target non esiste più (magari già terminato da un altro processo), si riprende semplicemente il chiamante senza fare nulla (**LDST(savedState)**: nota che qui si usa direttamente **savedState**, non **¤tProcess->p_s**, perché il chiamante — che *non* è il target in questo ramo — non ha ancora subito modifiche al proprio stato salvato). Se il target esiste, si chiama **terminateProcess** (che termina ricorsivamente lui e tutto il suo sotto-albero, si veda sopra). Se il chiamante ha terminato *se stesso*, **currentProcess** è già stato azzerato da **terminateProcess**, e si richiama direttamente lo scheduler; altrimenti (il chiamante ha terminato *un altro* processo, es. un fratello o un discendente) il chiamante stesso prosegue: si salva il suo stato e si riprende con **LDST**.

7.6.4 SYS3 — PASSEREN (P)

```

1  case PASSEREN: {
2      int *semAddr = (int *) savedState->reg_a1;

```

```

3   updateCPUTime();
4   copyState(&currentProcess->p_s, savedState);
5   (*semAddr)--;
6   if (*semAddr < 0) {
7       blockCurrentProcess(semAddr);
8   }
9   LDST(&currentProcess->p_s);
10  break;
11 }

```

Implementa la semantica classica del semaforo a **contatore negativo**: si decrementa sempre il valore; se il risultato è negativo significa che non c'era disponibilità, e il processo si blocca (`blockCurrentProcess`, che a sua volta chiama `scheduler()` e non ritorna qui). Se invece il valore resta ≥ 0 , il processo continua subito: si riprende con `LDST`. Da notare che lo stato viene salvato nel PCB (`copyState`) *prima* di decidere se bloccarsi, cosicché se il blocco avviene, `blockCurrentProcess` trova già uno stato coerente salvato in `p_s`.

7.6.5 SYS4 — VERHOGEN (V)

```

1  case VERHOGEN: {
2      int *semAddr = (int *) savedState->reg_a1;
3      updateCPUTime();
4      copyState(&currentProcess->p_s, savedState);
5      (*semAddr)++;
6      if (*semAddr <= 0) {
7          pcb_t *unblocked = removeBlocked(semAddr);
8          if (unblocked) {
9              unblocked->p_semAdd = NULL;
10             insertProcQ(&readyQueue, unblocked);
11         }
12     }
13     LDST(&currentProcess->p_s);
14     break;
15 }

```

Incrementa il semaforo; se il valore risultante resta ≤ 0 , significa che c'era (almeno) un processo in coda: lo si rimuove dalla ASL (`removeBlocked`, che restituisce il primo in coda in ordine FIFO) e lo si reinserisce nella ready queue. Il chiamante della V *non si blocca mai*: dopo l'eventuale sblocco altrui, riprende sempre subito con `LDST`.

7.6.6 SYS5 — DOIO

```

1  case DOIO: {
2      int *commandAddr = (int *) savedState->reg_a1;
3      int  commandValue = (int) savedState->reg_a2;
4
5      unsigned int offset = (unsigned int)commandAddr - START_DEVREG;
6      int line      = (int)(offset / 0x80) + 3;
7      int dev       = (int)((offset % 0x80) / 0x10);
8      int subword   = (int)((offset % 0x10) / WORDLEN);
9
10     int semIdx = (line == 7)
11     ? ((subword == 3) ? TERM_TX_SEM(dev) : TERM_RX_SEM(dev))
12     : DEV_SEM_BASE(line, dev);
13
14     updateCPUTime();
15     copyState(&currentProcess->p_s, savedState);
16
17     *commandAddr = commandValue;
18     devSems[semIdx]--;
19     blockCurrentProcess(&devSems[semIdx]);
20     break;
21 }

```

Avvia un'operazione di I/O e blocca *sempre* il processo chiamante, perché ogni operazione di I/O è per natura asincrona e si conclude con un interrupt di completamento gestito da `interrupts.c`.

1. Calcolo del **semaforo giusto** a partire dall'indirizzo del registro di comando passato dal chiamante (`commandAddr`): sottraendo la base `START_DEVREG` si ottiene un offset, da cui si ricavano linea di interrupt (`line`, ogni linea occupa `0x80` byte), numero di device sulla linea (`dev`, ogni device occupa `0x10` byte) e la “subword” dentro il blocco di registri del device (`subword`, ogni registro è una word da 4 byte). Per la linea terminale (`line == 7`), che ha 4 registri per device (status/comando di ricezione, poi status/comando di trasmissione), la subword 3 identifica il registro di comando *trasmissione* → semaforo TX, altrimenti si usa il semaforo RX.
2. Si aggiorna la contabilità del tempo CPU e si salva lo stato corrente.
3. **Si scrive il comando sul registro del device** (`*commandAddr = commandValue`): questo è l'istante in cui l'I/O viene effettivamente avviato a livello hardware.
4. Si decrementa il semaforo del device e si blocca il processo su di esso (`blockCurrentProcess`).

Assenza di race condition: tra la scrittura del comando e il blocco del processo il Nucleus gira sempre con gli interrupt disabilitati (il Nucleus in generale processa le eccezioni con interrupt mascherati fino al prossimo `WAIT/LDST`); anche se il device completasse l'operazione “istantaneamente”, l'interrupt corrispondente resterebbe *pending* e verrebbe servito solo dopo che lo scheduler ha eseguito `WAIT` o è ripartito con un altro processo — non si perde mai il segnale di completamento.

7.6.7 SYS6 — GETTIME

```

1 case GETTIME: {
2     updateCPUtime();
3     savedState->reg_a0 = (unsigned int) currentProcess->p_time;
4     LDST(savedState);
5     break;
6 }
```

Aggiorna la contabilità (per includere anche il tempo trascorso fino a questo istante nella syscall stessa) e restituisce in `a0` il tempo CPU totale accumulato dal processo chiamante. Nessun blocco: si riprende subito con `LDST(savedState)` (usa direttamente lo stato originale letto da `BIOSDATAPAGE`, dato che non serve passare per il PCB in questo caso semplice).

7.6.8 SYS7 — CLOCKWAIT

```

1 case CLOCKWAIT: {
2     updateCPUtime();
3     copyState(&currentProcess->p_s, savedState);
4     devSems[PSEUDOCLOCK_SEM]--;
5     softBlockCount++;
6     blockCurrentProcess(&devSems[PSEUDOCLOCK_SEM]);
7     break;
8 }
```

Blocca il processo chiamante fino al prossimo tick dello pseudo-clock (ogni `PSECOND`, 100 ms). A differenza di `PASSEREN` generica, qui `softBlockCount++` viene incrementato **esplicitamente prima** di chiamare `blockCurrentProcess`: dato che il semaforo pseudo-clock è per definizione escluso da `isDeviceSemaphore` (per evitare doppio conteggio quando *altri* punti del codice bloccano su semafori generici), qui il chiamante si assume esplicitamente la responsabilità di incrementare il contatore, così che lo scheduler possa comunque distinguere correttamente “sto aspettando il prossimo tick” da un deadlock.

7.6.9 SYS8 — GETSUPPORTPTR

```

1 case GETSUPPORTPTR: {
2     savedState->reg_a0 = (unsigned int) currentProcess->p_supportStruct;
3     LDST(savedState);
4     break;
5 }
```

Restituisce semplicemente il puntatore alla `support_t` del processo chiamante (NULL se non ne ha una). È la syscall con cui, in Fase 3, il Pager e il general exception handler recuperano la propria support structure appena vengono attivati dal pass-up.

7.6.10 SYS9 — GETPROCESSID

```

1 case GETPROCESSID: {
2     int wantParent = (int) savedState->reg_a1;
3     savedState->reg_a0 = (wantParent == 0)
4         ? (unsigned int) currentProcess->p_pid
5         : (currentProcess->p_parent ? (unsigned int) currentProcess->p_parent->p_pid : 0);
6     LDST(savedState);
7     break;
8 }

```

Se `a1 == 0` restituisce il PID del chiamante stesso; se `a1 != 0` restituisce il PID del *padre* del chiamante, oppure 0 se il chiamante non ha un padre (è il processo radice).

7.6.11 SYS10 — YIELD

```

1 case YIELD: {
2     updateCPUTime();
3     copyState(&currentProcess->p_s, savedState);
4     yieldedProcess = currentProcess;
5     currentProcess = NULL;
6     scheduler();
7     break;
8 }

```

Il processo chiamante cede volontariamente la CPU. *Non* viene reinserito subito nella ready queue: viene invece registrato in `yieldedProcess`, e sarà lo scheduler (passo 0, discusso sopra) a decidere se rimetterlo in coda (perché esiste un processo pronto di priorità pari o superiore) oppure farlo ripartire immediatamente con una nuova time slice (se è comunque il candidato più prioritario). Questo meccanismo indiretto garantisce che uno `YIELD` *ceda davvero* la CPU quando ha senso farlo, invece di rientrare subito in `insertProcQ` e ripescarsi da solo in coda per un caso limite.

7.7 passUpOrDie — il meccanismo di pass-up-or-die

```

1 static void passUpOrDie(int exceptionType) {
2     if (!currentProcess || !currentProcess->p_supportStruct) {
3         updateCPUTime();
4         terminateProcess(currentProcess);
5         currentProcess = NULL;
6         scheduler();
7     } else {
8         support_t *sup = currentProcess->p_supportStruct;
9         copyState(&sup->sup_exceptState[exceptionType], (state_t *) BIOSDATAPAGE);
10        context_t *ctx = &sup->sup_exceptContext[exceptionType];
11        LDCXT(ctx->stackPtr, ctx->status, ctx->pc);
12    }
13 }

```

Il cuore del disaccoppiamento tra Nucleus e Support Level. Chiamato ogni volta che il Nucleus incontra un'eccezione che *non sa* gestire direttamente (page fault, program trap, o syscall con codice ≥ 1):

- **Caso “die”:** se non c'è un processo corrente, oppure il processo corrente *non ha* una support structure (`p_supportStruct == NULL`, tipico di un processo puramente Fase 2 come `test` in modalità pura, che non ha nessun livello superiore a cui rivolgersi), il processo viene semplicemente terminato con `terminateProcess` (che, ricordiamo, termina anche tutto il suo sotto-albero), e si richiama lo scheduler.

- **Caso “pass up”**: se invece il processo *ha* una support structure (una U-proc di Fase 3), lo stato salvato in BIOSDATAPAGE viene copiato nello slot corrispondente (`sup_exceptState[PGFAULTEXCEPT]` o `sup_exceptState[GENERALEXCEPT]`, a seconda del tipo passato come argomento), e si esegue un LDCXT con lo stack pointer, lo status e il PC salvati nel `context_t` corrispondente (`sup_exceptContext[...]`): è un vero salto (non-locale) a un gestore di livello Support (il Pager o il general exception handler, configurati in Fase 3 da `launchUproc`), su uno stack dedicato interamente separato da quello del Nucleus.

Questo è esattamente ciò che permette alla Fase 3 di essere costruita *sopra* il Nucleus senza toccarne il codice: il Nucleus si limita a “spedire” l’eccezione al livello giusto, senza sapere nulla di cosa significhino PGFAULTEXCEPT/GENERALEXCEPT per chi le riceve.

8 phase2/interrupts.c — gestione degli interrupt

8.1 Costanti e utility

```

1 #define INT_BITMAP_BASE 0x10000040
2 #define INT_BITMAP(line) (*((unsigned int *) (INT_BITMAP_BASE + ((line) - 3) * 0x4)))
3
4 #define DEV_REG_BASE(line, dev) \
5     ((unsigned int *) (START_DEVREG + ((line) - 3) * 0x80 + (dev) * 0x10))
6
7 #define DEV_STATUS(base) ((base)[0])
8 #define DEV_COMMAND(base) ((base)[1])
9
10 #define TERM_RECV_STATUS(base) ((base)[0])
11 #define TERM_RECV_COMMAND(base) ((base)[1])
12 #define TERM_TRANSM_STATUS(base) ((base)[2])
13 #define TERM_TRANSM_COMMAND(base) ((base)[3])
14
15 static int getHighestPriorityDevice(unsigned int bitmap) {
16     for (int i = 0; i < 8; i++) {
17         if (bitmap & (1u << i)) return i;
18     }
19     return -1;
20 }

```

`INT_BITMAP(line)` legge, per una data linea di interrupt (3–7), la bitmap hardware a 8 bit che indica quali degli 8 device su quella linea hanno un interrupt pendente. `DEV_REG_BASE(line, dev)` calcola l’indirizzo del blocco di registri di un device specifico (ogni linea occupa 0x80 byte, ogni device 0x10 byte all’interno della linea). Le macro `DEV_STATUS/DEV_COMMAND` e le quattro `TERM_*` sono semplici alias per interpretare quel blocco come registri (status, comando) di un device generico o, per i terminali, come due coppie indipendenti (ricezione, trasmissione). `getHighestPriorityDevice(bitmap)` scansiona i bit da 0 a 7 e ritorna l’indice del *primo* bit acceso (cioè il device a **priorità più alta**, dato che per convenzione hardware il bit meno significativo ha priorità maggiore); -1 se nessun bit è acceso (interrupt spurio).

8.2 interruptHandler — struttura generale

```

1 void interruptHandler(void) {
2     state_t *savedState = (state_t *) BIOSDATAPAGE;
3     unsigned int cause = savedState->cause;
4     unsigned int excCode = cause & CAUSE_EXCCODE_MASK;
5
6     cpu_t now;
7     STCK(now);
8     if (currentProcess != NULL) {
9         currentProcess->p_time += (now - startTOD);
10    }
11    startTOD = now;
12
13    if (excCode == 7u) { /* PLT */ ... }
14    if (excCode == 3u) { /* Interval Timer */ ... }
15    if (excCode >= 17u && excCode <= 21u) { /* device */ ... }

```

```

16     if (currentProcess != NULL) LDST(savedState);
17     else scheduler();
18 }
19

```

Legge *cause* e il suo *excCode*, aggiorna la contabilità del tempo CPU (stessa logica di *updateCPUTime*, qui reimplementata inline), e poi distingue tre grandi classi in base al valore di *excCode*: 7 (PLT), 3 (Interval Timer), 17-21 (le cinque linee hardware dei device esterni, che nel registro *cause* sono mappate a partire da 17 pur corrispondendo alle linee fisiche 3-7, da cui il “-14” visto più avanti). Se nessuno di questi rami restituisce prima (*return*), il codice finale ripristina *currentProcess* con *LDST* se esiste ancora, altrimenti chiama lo scheduler.

8.3 PLT — fine time slice (preemption)

```

1  if (excCode == 7u) {
2      setTIMER((cpu_t) NEVER);
3      if (currentProcess != NULL) {
4          copyState(&currentProcess->p_s, savedState);
5          currentProcess->p_s.status |= MSTATUS_MIE_MASK;
6          insertProcQ(&readyQueue, currentProcess);
7          currentProcess = NULL;
8      }
9      scheduler();
10     return;
11 }

```

Questo è il meccanismo che realizza la **preemption**: quando la time slice di 5 ms del processo corrente scade, il PLT genera questo interrupt. Il gestore:

1. **Disarma il PLT** caricandogli il valore massimo possibile (*NEVER* = 0x7FFFFFFF), così non genererà ulteriori interrupt finché lo scheduler non ne programma uno nuovo per il prossimo processo.
2. Se c’era un processo in esecuzione, ne salva lo stato (*copyState*), **si assicura che, alla prossima ripartenza, abbia gli interrupt abilitati** (forzando il bit *MSTATUS_MIE_MASK* nello *status* salvato — precauzione difensiva, dato che lo stato salvato dal firmware al momento dell’interrupt riflette lo *status precedente* all’interrupt stesso), e lo rimette nella ready queue (*insertProcQ*, che lo riposiziona secondo la sua priorità: un round-robin “puro” tra processi di pari priorità).
3. Richiama lo scheduler, che sceglierà il prossimo processo da eseguire (possibilmente lo stesso, se non c’è nessun altro pronto).

8.4 Interval Timer — tick dello pseudo-clock

```

1  if (excCode == 3u) {
2      LDIT(PSECOND);
3
4      pcb_t *p;
5      while ((p = removeBlocked(&devSems[PSEUDOCLOCK_SEM])) != NULL) {
6          p->p_semAdd = NULL;
7          p->p_s.reg_a0 = 0;
8          insertProcQ(&readyQueue, p);
9          softBlockCount--;
10     }
11
12     devSems[PSEUDOCLOCK_SEM] = 0;
13
14     if (currentProcess != NULL) LDST(savedState);
15     else scheduler();
16     return;
17 }

```

Ogni 100 ms (*PSECOND*) l’Interval Timer genera questo interrupt.

1. **Riarma subito il timer** per il prossimo tick (*LDIT(PSECOND)*).

2. **Sblocca in blocco** tutti i processi che erano in attesa sul semaforo pseudo-clock (chiunque avesse chiamato `CLOCKWAIT`): il ciclo `while` continua a chiamare `removeBlocked` finché la coda non è vuota, e per ciascun processo estratto: azzerare `p_semAdd`, imposta `a0 = 0` (valore di ritorno di `CLOCKWAIT`), lo inserisce nella ready queue, e decrementa `softBlockCount`.
3. **Azzerare esplicitamente** `devSems[PSEUDOCLOCK_SEM]`: a differenza di un semaforo normale (dove ogni V incrementa di 1), qui si forza il valore a 0 perché lo pseudo-clock ha semantica “broadcast”: *tutti* i processi in attesa vengono liberati contemporaneamente a ogni tick, non uno alla volta come farebbe una sequenza di V singole.
4. Come sempre, se c’era un processo in esecuzione lo si riprende direttamente (`LDST(savedState)`, nessun cambio di processo necessario); altrimenti si chiama lo scheduler (che magari ora ha nuovi processi pronti, appena svegliati).

8.5 Interrupt di device (linee 3–7, excCode 17–21)

```

1  if (excCode >= 17u && excCode <= 21u) {
2      int intLineNo = (int)(excCode - 14u);
3      unsigned int bitmap = INT_BITMAP(intLineNo);
4      int devNo = getHighestPriorityDevice(bitmap);
5
6      if (devNo < 0) {
7          if (currentProcess != NULL) LDST(savedState);
8          else scheduler();
9          return;
10     }
11
12     if (intLineNo == 7) {
13         /* terminale: TX poi RX, vedi sotto */
14     } else {
15         unsigned int *devBase = DEV_REG_BASE(intLineNo, devNo);
16         unsigned int savedStatus = DEV_STATUS(devBase);
17         DEV_COMMAND(devBase) = ACK;
18
19         int semIdx = DEV_SEM_BASE(intLineNo, devNo);
20         if (devSems[semIdx] < 0) {
21             devSems[semIdx]++;
22             pcb_t *unblocked = removeBlocked(&devSems[semIdx]);
23             if (unblocked != NULL) {
24                 unblocked->p_s.reg_a0 = savedStatus;
25                 unblocked->p_semAdd = NULL;
26                 insertProcQ(&readyQueue, unblocked);
27                 softBlockCount--;
28             }
29         }
30     }
31
32     if (currentProcess != NULL) LDST(savedState);
33     else scheduler();
34     return;
35 }

```

1. `intLineNo = excCode - 14`: converte il codice *cause* (17–21) nella linea hardware effettiva (3–7: `IL_DISK`, `IL_FLASH`, `IL_ETHERNET`, `IL_PRINTER`, `IL_TERMINAL`).
2. Legge la bitmap dei device con interrupt pendente su quella linea, e sceglie `devNo` col bit più basso acceso (priorità più alta). Se nessun bit è acceso, l’interrupt è spurio: si riprende/schedula senza fare nulla.
3. **Per device diversi dal terminale** (linee 3–6: disk, flash, ethernet, printer): si legge lo **status prima** di inviare l’ACK. Questo ordine è *cruciale*: il registro di status del device viene tipicamente sovrascritto o azzerato nel momento in cui si invia il comando di ACK, quindi se si invertisse l’ordine il valore di status andrebbe perso e il processo risvegliato non saprebbe come è terminata la sua operazione di I/O. Dopo aver salvato lo status, si scrive ACK nel registro di comando (riconoscimento dell’interrupt), poi: se il semaforo del device era negativo (c’era almeno un processo in attesa), lo si incrementa e si sblocca il primo processo

in coda (`removeBlocked`), impostandone `a0` allo status letto (valore di ritorno di `DOI0`) e decrementando `softBlockCount`. Se il semaforo *non* era negativo (nessuno in attesa — caso anomalo dato che `DOI0` blocca sempre, ma gestito difensivamente), non si fa nulla.

4. Al termine, come sempre, si riprende il processo corrente o si schedula.

8.5.1 Il caso speciale del terminale (linea 7)

```

1 unsigned int *termBase = DEV_REG_BASE(intLineNo, devNo);
2 unsigned int txStatus = TERM_TRANSM_STATUS(termBase) & 0xFFu;
3 unsigned int rxStatus = TERM_RECV_STATUS(termBase) & 0xFFu;
4
5 if (txStatus == OKCHARTRANS) {
6     unsigned int savedStatus = TERM_TRANSM_STATUS(termBase);
7     TERM_TRANSM_COMMAND(termBase) = ACK;
8     int semIdx = TERM_TX_SEM(devNo);
9     if (devSems[semIdx] < 0) {
10         devSems[semIdx]++;
11         pcb_t *unblocked = removeBlocked(&devSems[semIdx]);
12         if (unblocked != NULL) {
13             unblocked->p_s.reg_a0 = savedStatus;
14             unblocked->p_semAdd = NULL;
15             list_add_tail(&unblocked->p_list, &readyQueue);
16             softBlockCount--;
17         }
18     }
19 }
20
21 if (rxStatus == CHARRECV) {
22     unsigned int savedStatus = TERM_RECV_STATUS(termBase);
23     TERM_RECV_COMMAND(termBase) = ACK;
24     int semIdx = TERM_RX_SEM(devNo);
25     if (devSems[semIdx] < 0) {
26         devSems[semIdx]++;
27         pcb_t *unblocked = removeBlocked(&devSems[semIdx]);
28         if (unblocked != NULL) {
29             unblocked->p_s.reg_a0 = savedStatus;
30             unblocked->p_semAdd = NULL;
31             insertProcQ(&readyQueue, unblocked);
32             softBlockCount--;
33         }
34     }
35 }

```

Un terminale è in realtà **due sotto-device indipendenti**: trasmissione (TX, scrittura di caratteri) e ricezione (RX, lettura di caratteri), ciascuno con il proprio status/comando e il proprio semaforo dedicato (`TERM_TX_SEM`/`TERM_RX_SEM`). Un singolo interrupt sulla linea 7 può quindi segnalare il completamento di *una* delle due operazioni (o, in teoria, di entrambe se coincidono): il codice controlla **prima** lo status di trasmissione (`OKCHARTRANS` = 5: carattere trasmesso con successo) e **poi** quello di ricezione (`CHARRECV` = 5: carattere ricevuto), gestendo ciascuno in modo indipendente con la stessa logica (salva status, ACK, eventuale sblocco del processo in attesa) vista sopra per i device generici. **TX ha priorità su RX**: se entrambi gli status indicano un evento completato nello stesso interrupt, si processa prima la trasmissione. Questo ordine evita che un carattere ricevuto “scavalchi” silenziosamente un carattere appena trasmesso in situazioni di traffico bidirezionale rapido, riducendo il rischio di perdita di caratteri.

(Nota implementativa minore: nel ramo TX lo sblocco usa `list_add_tail(&unblocked->p_list, &readyQueue)` anziché `insertProcQ`: essendo la ready queue ordinata per priorità tramite `p_prio`, e i processi risvegliati da un device tipicamente non hanno priorità speciali diverse da quella con cui erano stati creati, l'effetto pratico coincide quasi sempre con un `insertProcQ`; il ramo RX usa invece coerentemente `insertProcQ`.)

9 Il ciclo completo di una operazione di I/O — riepilogo narrativo

È lo scenario più chiesto all'orale: sapere raccontare, passo per passo, cosa succede quando un processo chiede un'operazione di I/O.

1. Un processo esegue `SYSCALL(DOIO, indirizzoComando, comando, 0)`. Questo genera un'eccezione `ecall`, che il firmware intercetta salvando lo stato in `BIOSDATAPAGE` e saltando a `exceptionHandler` (l'indirizzo registrato nel Pass Up Vector).
2. `exceptionHandler` riconosce l'`ExcCode` come `ECALL`, avanza il PC di una word, e chiama `syscallHandler`.
3. `syscallHandler` riconosce `sysCode == DOIO`, calcola il semaforo giusto dall'indirizzo del registro di comando, **scrive il comando sul device** e poi chiama `blockCurrentProcess`: il processo viene inserito nella ASL sul semaforo del device e `currentProcess` torna NULL.
4. Si richiama `scheduler()`. Se non ci sono altri processi pronti ma qualcuno è soft-blocked (come il processo appena bloccato), lo scheduler esegue `WAIT()`: la CPU virtuale si ferma in attesa di un interrupt, con tutti gli interrupt (tranne il PLT) abilitati.
5. Il device, terminata l'operazione richiesta, genera un **interrupt** sulla propria linea. Il firmware salva lo stato e salta di nuovo a `exceptionHandler`, che lo riconosce (bit 31 di `cause`) e delega a `interruptHandler`.
6. `interruptHandler` legge la bitmap della linea, individua il device, **legge lo status prima di fare l'ACK**, incrementa il semaforo del device e sblocca il processo in attesa con `removeBlocked`, impostandogli `a0` allo status letto.
7. Il processo risvegliato torna nella ready queue. Alla sua prossima esecuzione (dispatch dello scheduler), riprenderà dall'istruzione *successiva* alla `ecall` della `DOIO`, trovando in `a0` lo status dell'operazione appena conclusa.

Nessuna race condition è possibile perché tra l'emissione del comando (passo 3) e il blocco effettivo del processo, il Nucleus sta ancora gestendo un'eccezione con gli interrupt mascherati: anche se il device completasse l'operazione istantaneamente, l'interrupt resterebbe pending e verrebbe processato solo dopo che il controllo è tornato a un punto sicuro (`WAIT` o la ripresa di un altro processo).

10 Il test driver ufficiale: `phase2/p2test.c`

Come `p1test.c` per la Fase 1, `p2test.c` **non è codice originale del progetto**: è il test driver standard del Nucleus (compilato solo nel target `phase2`; fornisce la funzione `test()` che `initial.c::main` lancia come primo processo). È volutamente “contorto” (lo dice il commento stesso in testa al file, “*This is pretty convoluted code, so good luck!*”) perché il suo scopo è mettere sotto stress *ogni* syscall e meccanismo del Nucleus in un'unica esecuzione. Non viene analizzato riga per riga (è quasi 800 righe, in gran parte ripetitive), ma è utile sapere cosa dimostra ciascun sotto-processo, perché il professore può chiedere “spiega cosa succede quando gira `p2test`”.

Processo	Ruolo	Cosa dimostra
<code>test</code> (“p1”)	radice	Fa 3 V e 3 P consecutivamente, verificandone il valore e incrementandolo. Gli stati iniziali di tutti i semafori sono crescenti, ciascuno nella propria sequenza con <code>CREATE</code> .

p2	sincronizzazione + tempo	Si blocca f VERHOGEN(sem_star PID (GETPROCESSID stesso semaforo con confronta GETTIME d care che la contabil e coerente (mai più Cicla su CLOCKWAIT venga dopo circa u che durante l'attesa (GETTIME) resti coer Crea una <i>seconda in</i> codice, PCB diverso ta; quando il padre stesso), il Nucleus d figlio: verifica termi È l'unico processo c (pFiveSupport) anc memoria virtuale: s simo generico). Gen a un indirizzo invali un accesso fuori kus → ancora p5gen, ch poi una syscall dal fault (p5mm). Dimo correttamente le div text (GENERALEXCEP nella sua support_t Esegue una sys (SYSCALL(1,...)) struct: il Nucleus d (ramo "die" di pass al print successivo, Stesso principio di indirizzo non valido i stesso ramo "die" pe to da un vero progr Costruisce un albero figli, 4 nipoti) e lo ri ca non solo che term mente un sotto-alber la radice muore, ma ti vengano <i>davvero</i> (la ripetizione fallire perdita di risorse). p9 crea p10 e si blo <i>dre</i> esplicitamente p ppid), verificando il chiamante non coin questo documento:
p3	pseudo-clock	
p4	terminazione a cascata	
p5 (con support_t)	pass-up-or-die <i>con</i> support struct	
p6	pass-up-or-die <i>senza</i> support struct (syscall)	
p7	pass-up-or-die <i>senza</i> support struct (program trap)	
p8root + child1/child2 + 4 foglie	sotto-alberi profondi	
p9/p10	terminazione di un processo diverso da sé	

Ogni sotto-processo stampa messaggi di progresso sul terminale 0 tramite una funzione `print` interna (analogo a `termprint` di `p1test.c`, ma qui implementata sopra una vera `DOIIO`, dato che il Nucleus è già disponibile), protetta da un semaforo di mutua esclusione (`sem_term_mut`) per evitare che più processi scrivano contemporaneamente e mescolino i caratteri sullo stesso terminale — un piccolo promemoria pratico che anche una risorsa condivisa “interna” come il terminale di debug va protetta con un semaforo.

`p2test_OLD.c`. Nella cartella `phase2/` è presente anche un file `p2test_OLD.c`: è una versione precedente dello stesso test driver, **non referenziata da nessun target CMake** (si veda `CMakeLists.txt`: solo `phase2/p2test.c` compare nell’elenco sorgenti del target `phase2`) e quindi non più compilata né eseguita; resta nel repository solo come riferimento storico.